FIUS-MoveSense: Ultrasonic Movement Detection Using Smart Ultrasonic Sensors (FIUS) and Machine Learning

Information Technology(M. Eng.)
Machine Learning
Name: Md Maruf hossain
Matriculation No: 1390272
Email: maruf.hossain@stud.fra-uas.de

Information Technology(M. Eng.)
Machine Learning
Name: *Md Jamal Uddin*
Matriculation No: 1387201
Email: md.uddin2@stud.fra-uas.de

Information Technology(M. Eng.)
Machine Learning)
Name: Mousumi Parvin Tonny
Matriculation No: 1393224
Email: mousumi.tonny@stud.fra-uas.de

Information Technology(M. Eng.)
Machine Learning
Name: *Md. Shamsir Doha*
Matriculation No: 1344011
Email: mohammad.doha@stud.fra-uas.de

***Abstract*— This report describes FIUS-Move Sense, a system designed to detect movement using ultrasonic sensors together with machine learning methods. The system works by analyzing ultrasonic echo signals generated by FIUS sensors to identify whether an object in the monitored area is moving or stationary. During each scan, the sensor collects around 50,000 sample data points, which represent the amplitudes of the reflected ultrasonic signal. To make this data useful for classification, several signal-processing and feature-extraction techniques are applied in order to convert the raw signal into meaningful statistical features. These features are then used to train and evaluate different supervised machine-learning models, including Random Forest, Support Vector Machine (SVM), k-Nearest Neighbors (KNN), and Logistic Regression. The experimental results indicate that the Random Forest model produced the highest performance, reaching an accuracy of more than 90%. Overall, the study shows that combining ultrasonic signal analysis with machine learning can be an effective approach for reliable movement detection in intelligent sensing applications. [1][2]. The proposed system achieved an accuracy of 90.5% using a Random Forest classifier and an F1-score of 0.8896 on a dataset of 25,490 ultrasonic scans. The system was further validated using real-world test recordings, successfully detecting movement directions such as approaching, moving away, and lateral motion.**

Keywords*— *Ultrasonic sensing, Confusion matrix, feature extraction, Movement detection, Machine learning, FIUS sensor, Signal processing, Random Forest, Bessel filter, binary classification, support vector machine.

I. Introduction

Movement detection plays an important role in many intelligent systems, including robotics, security monitoring, smart home technologies, and industrial automation. Many traditional motion detection systems rely on cameras or infrared sensors. However, these approaches can be affected by factors such as lighting conditions or environmental changes. Ultrasonic sensors provide a reliable alternative because they detect objects by transmitting sound waves and measuring the reflected echoes. Since this method does not depend on visible light, ultrasonic sensing can work effectively in different lighting conditions and environments.

The FIUS smart ultrasonic sensor produces echo signals that correspond to reflections from objects located in front of the sensor. When an object moves, the pattern of these reflected signals changes between successive scans. By analyzing these variations in the signal, machine learning techniques can be used to determine whether movement has occurred. In this project, a machine learning pipeline is developed to process the ultrasonic signal data and classify the state of the observed object as either moving or stationary.

Machine learning offers a framework in which algorithms can learn patterns directly from data and improve their performance over time without requiring explicitly programmed instructions. By analyzing input data, these models are able to make predictions or decisions based on patterns learned from previous observations. Because of this ability to learn and adapt, machine learning has become a powerful approach for enabling automated systems that require only limited human intervention.

Machine learning models rely heavily on the quality and reliability of the data used as input. In sensor-based perception systems, correctly interpreting environmental measurements is crucial for identifying and classifying movement patterns. In this project, smart ultrasonic sensors are used to collect distance measurements that reflect the motion of objects in the surrounding environment. One of the main challenges is enabling machine learning algorithms to interpret the raw sensor signals correctly and distinguish meaningful movement patterns from noise or irrelevant variations. Raw ultrasonic measurements may include fluctuations, measurement inaccuracies, or interference from the environment, all of which can reduce the performance of the model. For this reason, preprocessing steps such as signal filtering, normalization, and feature extraction are applied to improve the accuracy, efficiency, and robustness of the machine learning models used for movement detection.

This project investigates movement detection using ultrasonic sonar signals together with machine learning techniques. Ultrasonic pulses at 40 kHz are transmitted by a sensor system connected to a Red Pitaya platform, and the reflected echoes are recorded as high-resolution digital time-series signals containing approximately 50,000 samples. The

collected signals are first preprocessed and then analyzed through feature extraction methods. This includes frequency-domain analysis using Fourier Transform techniques to identify signal characteristics that may indicate movement. The extracted features are subsequently used to train and evaluate different machine learning models for movement classification. The system performs a binary classification task to distinguish between moving and stationary objects. In addition, the project also explores whether it is possible to detect the direction of movement, specifically whether an object is approaching the sensor or moving away from it. Model performance is evaluated using confusion matrices and other common classification metrics in order to assess the reliability of the proposed movement detection approach. The main objectives of this project are:

(1) To develop a complete machine learning pipeline that processes raw sensor data and produces movement predictions.

(2) To design features that capture movement patterns regardless of the object's distance from the sensor.

(3) To compare several classification algorithms and optimize the decision threshold for improved real-world performance.

(4) To investigate movement direction detection as an additional feature for extra credit.

Overall, in this project, an ultrasonic movement detection system called FIUS-MoveSense is developed using a FIUS ultrasonic sensor and a Red Pitaya ADC board. The system captures ultrasonic echo signals and processes them using signal processing and machine learning techniques. A total of 38 features is extracted from each scan and used to train several machine learning models for movement classification.

Several machine learning models including Random Forest, Support Vector Machine, K-Nearest Neighbors, and Logistic Regression are evaluated. Experimental results show that the Random Forest classifier achieves the best performance, demonstrating the effectiveness of the proposed system for detecting human movement using ultrasonic sensing.

II. THEORETICAL BACKGROUND

This section provides an overview of the theoretical background and main components used in the proposed system. The movement detection system is developed using a combination of hardware and software elements that work together to analyze ultrasonic sensing data. The hardware setup includes the Red Pitaya STEMlab board and an ultrasonic sensing module, which are used to generate and capture the echo signals. In addition to the hardware components, this section also introduces the programming environment used for signal processing and data analysis. Furthermore, the machine learning techniques applied to build and evaluate the movement classification model are described.

A. Hardware Setup (Red Pitaya + ultrasonic sensor)

The Red Pitaya STEMlab board serves as the main data acquisition platform used in this project. It is equipped with high-performance Analog-to-Digital Converters (ADC) and Digital-to-Analog Converters (DAC), which allow high-bandwidth signal acquisition and processing. Red Pitaya is an open-source hardware and software platform that also provides a user-friendly web interface for data visualization and remote control of the system. The board includes several general-purpose input/output (GPIO) ports, an RJ45 Ethernet connector, and USB connectivity, which makes it easy to integrate with external sensing modules. In this project, the Red Pitaya board is connected to an ultrasonic sensor that captures movement data relative to the sensor position. This includes movement toward the sensor, movement away from it, sideways movement, and situations where no movement occurs. Although the Red Pitaya platform is commonly used in radio-frequency applications such as software-defined radio systems, it is used in this study to acquire and process ultrasonic signals within a frequency range of approximately 30 kHz to 50 kHz for movement detection.

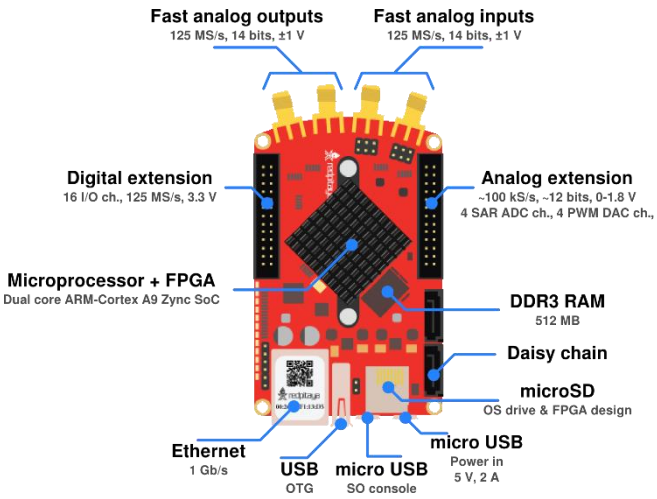

Figure1: Red pitaya STEMlab board configuration[3]

The Fully Integrated Ultrasonic Sensor (FIUS) is a compact and efficient sensing device designed for object detection and distance measurement. Ultrasonic sensors work by transmitting high-frequency sound waves and detecting the echoes that are reflected from nearby objects. The distance between the sensor and the object can then be determined using the time-of-flight principle, which measures how long it takes for the ultrasonic wave to travel to the object and return to the sensor. Because of their reliability and accuracy, ultrasonic sensors are commonly used in applications such as robotics, automation systems, and proximity sensing. In this project, the FIUS sensor is used to capture ultrasonic echo signals, which are later processed and analyzed to detect and classify movement.

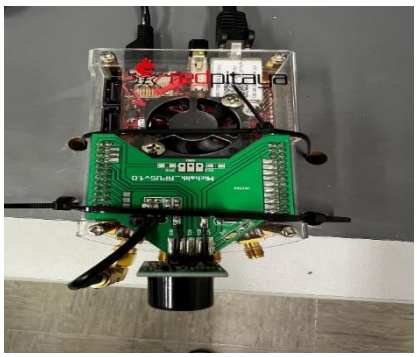

Figure 2: Red pitaya board's hardware and sensors

After calculating this data, first echo can be easily detected for the object and then the distance of that object can be measured.

B. Echo Detection

Echo detection plays a key role in determining whether an object is present in front of an ultrasonic sonar sensor. When the sensor emits a high-frequency ultrasonic signal, the sound wave travels through the air until it encounters an object and is reflected back toward the sensor. The returned reflection is received as an echo signal. By analyzing this returned signal, it is possible to determine the moment at which the first echo arrives. In this project, the first significant peak in the received signal is identified as the echo produced by the detected object. This peak indicates the presence of the object and serves as an important reference for subsequent signal processing and movement classification..

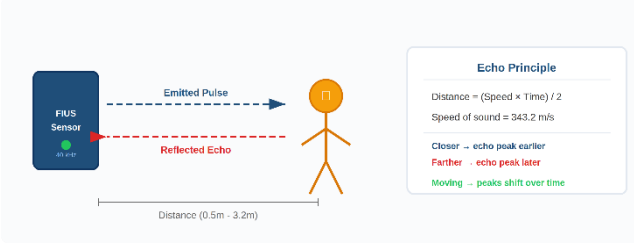

Figure 3: Ultrasonic pulse echo measuring process

As illustrated in Figure 3, the ultrasonic sensor emits a pulse which travels through the air and reflects from nearby objects. The reflected signal is captured by the receiver, and the time delay between transmission and reception provides information about the distance of the object. When the object moves, the position of the echo signal changes over time.

The detected echo signal is then used for further preprocessing and feature extraction before being applied to the machine learning classification model. The time-of-flight formula is: Distance = (Speed of Sound × Time) / 2, where the speed of sound in air is approximately 343.2 m/s.[8]

C. First Fourier Transform(FFT)

In this project, the Fast Fourier Transform (FFT) is applied to convert the acquired ultrasonic signals from the time domain into the frequency domain. The Red Pitaya server processes the captured data from the analog-to-digital converter (ADC) and outputs the signal in a discrete numerical format that can be used for further analysis. By transforming the signal into the frequency domain, it becomes easier to examine the spectral components of the ultrasonic waves. This frequency-based representation helps identify important characteristics of the signal that may be associated with object movement.

The Fast Fourier Transform (FFT) decomposes a signal into its individual frequency components, making it possible to identify the dominant frequencies present in the measured data. This method is commonly used in signal processing tasks such as system monitoring, fault detection, and condition analysis. In this study, an autocorrelation function is first applied to the raw ADC signal. After selecting a portion of the signal and applying a window function, the FFT is computed to obtain the power spectral density (PSD). The resulting spectral features are then used as input for the machine learning model, which classifies different movement patterns detected by the ultrasonic sensor.

D. Signal Acquisition and Digital Conversion

The data collected from the ultrasonic sensor is initially in analog form, representing the raw signal measurements. The Analog-to-Digital Converter (ADC) on the Red Pitaya board samples this high-frequency signal and converts it into a digital format that can be processed by the system. After the signal has been digitized, additional signal analysis can be performed using methods such as the Fast Fourier Transform (FFT) to study the frequency components of the ultrasonic waves. The resulting digital data is then used as input for machine learning algorithms, including Multilayer Perceptron (MLP) and Support Vector Machine (SVM), which are applied to classify the movement patterns detected by the sensor.

E. Bessel Filter

The Bessel filter is a type of linear filter that is known for its maximally flat group delay and nearly linear phase response. These characteristics allow the filter to preserve the shape of signals in the time domain while reducing unwanted noise. This property is especially important in ultrasonic signal processing, where maintaining the original waveform structure is necessary for accurate signal interpretation. By minimizing phase distortion, the Bessel filter can reduce high-frequency noise without significantly changing the waveform of the signal. In this project, the Bessel filter is applied during the signal preprocessing stage to improve the quality of the measured data before performing feature extraction and machine learning–based movement classification [4].

F. Support Vector Machine (SVM)

Support Vector Machines (SVMs) are supervised learning algorithms that are widely used for both classification and regression problems. The main goal of an SVM is to find an optimal hyperplane that separates data points from different classes while maximizing the margin between them. A larger margin generally improves the model's ability to generalize new, unseen data. Because of this property, SVMs are often effective when working with high-dimensional datasets, where the number of features can be relatively large.

When the data cannot be separated using a linear boundary, Support Vector Machines (SVM) use kernel functions to transform the input data into a higher-dimensional feature space where linear separation becomes possible. Several kernel functions are commonly used for this purpose, including the polynomial kernel, radial basis function (RBF) kernel, and the sigmoid kernel. Because of their robustness and strong ability to generalize, SVM models have been widely applied in many fields such as signal processing, pattern recognition, and biomedical data analysis [5]. In this project, SVM is used as one of the classification methods to distinguish different movement patterns based on the features extracted from the ultrasonic signals.

G. Random Forest Classifier

Random Forest is an ensemble learning algorithm that builds multiple decision trees during the training phase and determines the final prediction based on the majority vote of these trees. This method belongs to the family of bagging (bootstrap aggregating) techniques, where each tree is trained

using a randomly selected subset of the training data. In addition, during the tree construction process, a random subset of features is evaluated at each split. The combination of random data sampling and feature selection helps reduce the correlation between individual trees and improves the model's ability to generalize to new data.

One of the main advantages of Random Forest is its strong resistance to overfitting compared to a single decision tree. While an individual decision tree may learn noise present in the training data, combining the predictions from many trees helps reduce this effect. As a result, the overall model becomes more stable and reliable. In addition, Random Forest is capable of capturing complex and non-linear relationships between input features and target variables. This makes the algorithm particularly suitable for real-world datasets, where such relationships often occur.

Another useful feature of the Random Forest algorithm is its ability to estimate feature importance. By analyzing how much each feature contributes to reducing impurity during the tree-building process, the model can generate a ranking of the most influential features in the dataset. This capability is especially helpful for understanding which signal characteristics play the most important role in detecting movement.

In this project, the Random Forest algorithm was used as one of the classification models to analyze ultrasonic sensor data for movement detection. Among the four classifiers evaluated in the study, the Random Forest model achieved the highest classification accuracy. This result demonstrates its strong ability to distinguish between different movement states and highlights its effectiveness for signal-based classification tasks of this type.[11]

H. Confusion Matrix

A confusion matrix is a widely used evaluation tool for measuring the performance of classification models. It compares the labels predicted by the model with the actual labels in the dataset, providing a clear overview of the model's predictions [6]. Unlike simple accuracy measures, a confusion matrix also reveals the different types of classification errors made by the model. This information helps in understanding the strengths and weaknesses of the classifier and provides deeper insight into its overall performance. [7]

The confusion matrix consists of four main components:

True Positives (TP): Instances where the model correctly predicts a specific movement class.

True Negatives (TN): Instances where the model correctly identifies that a sample does not belong to a particular movement class.

False Positives (FP): Cases where the model incorrectly predicts a movement class when it is not present.

False Negatives (FN): Cases where the model fails to detect the correct movement class.

In this project, confusion matrices are used to evaluate the performance of the machine learning models in classifying the different movement patterns detected by the ultrasonic sensor. Besides providing a visual representation of the classification results, the confusion matrix also serves as the basis for calculating several important evaluation metrics. These metrics include accuracy, precision, recall, and F1-score, which together provide a more comprehensive understanding of the classifier's overall performance and effectiveness:

Accuracy measures how many predictions are correctly classified compared to the total number of predictions made by the model.

Precision measures the ratio of correctly predicted positive samples to the total predicted positive samples, indicating the exactness of the model.

Recall represents the ratio of correctly predicted positive samples to all actual positive samples, reflecting the model's ability to detect relevant cases.

F1-score is the harmonic means of precision and recall, providing a balanced evaluation metric when both false positives and false negatives are important.

III. ENVIRONMETN SETUP

The development and implementation of this project were carried out on a Windows-based system using Python 3.12 as the main programming language. To maintain consistency and prevent dependency conflicts, a dedicated Python virtual environment was created for the project. The development work was performed using Visual Studio Code (VS Code) as the primary code editor, along with command-line tools for running scripts and managing software packages. Git was used for version control, enabling collaborative development and systematic tracking of code changes throughout the project.

The ultrasonic sensor signals were collected using a Red Pitaya STEMlab board connected to a Fully Integrated Ultrasonic Sensor (FIUS). The Red Pitaya board performed the analog-to-digital conversion (ADC) of the ultrasonic signals and stored the captured measurements in CSV format. These data files were then imported into the Python environment for further processing and analysis.

Several Python libraries were used to perform signal processing, machine learning, data handling, and visualization tasks in this project. NumPy was used for numerical computations and for manipulating the signal data arrays. SciPy provided a range of signal processing functions, including the Bessel filter, Hilbert transform, and Fast Fourier Transform (FFT). For the machine learning component, the scikit-learn library was used to implement classification algorithms such as Random Forest, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Logistic Regression. In addition, tools such as GridSearchCV were applied for hyperparameter optimization, while StandardScaler was used to normalize the extracted features before training the models.

Data manipulation and dataset preparation were carried out using the pandas library, which enabled efficient handling of CSV files and feature datasets. The trained machine learning models were saved and later loaded using joblib, allowing the models to be reused during the evaluation stage. For visualization of the experimental results, including confusion matrices and signal plots, the Matplotlib library was used.

All required libraries were installed and managed within the Python virtual environment to ensure a controlled and

reproducible development setup. This configuration supported an efficient workflow that included data preprocessing, signal processing, feature extraction, model training, evaluation, and visualization of the experimental results.

Component	Description
Operating System	Windows
Programming Language	Python 3.12
Development Environment	Visual Studio Code (VS Code)
Version Control	GitHub
Data Acquisition Hardware	Red Pitaya STEMlab board
Sensor	Fully Integrated Ultrasonic Sensor (FIUS)
Machine Learning Library	Scikit-learn
Signal Processing Libraries	NumPy, SciPy
Data Handling	Pandas
Model Serialization	Joblib
Visualization	Matplotlib
Environment Management	Python Virtual Environment

Table: Hardware and Software Environment Used for the Project

IV. System Architecture

A. Functional Diagram

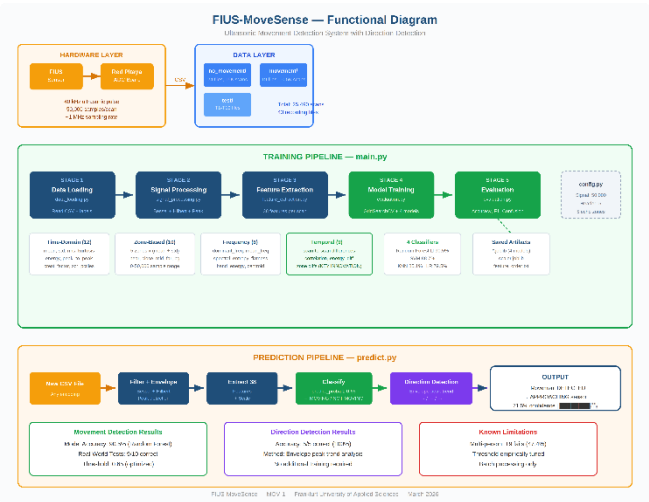

Figure 4: Overall architecture of the FIUS-MoveSense system.

Figure 4 shows the complete architecture of the proposed system. The FIUS ultrasonic sensor emits acoustic pulses while the Red Pitaya board captures the reflected echo signals. These signals are stored and processed in the data processing pipeline where features are extracted, and machine learning models are trained.

B. Pipeline Overview

The system is organized as a **six-stage processing pipeline**, where each stage is implemented as a separate Python module. The overall workflow is coordinated by **main.py**, which runs all stages of the pipeline in sequence. In addition, the file **predict.py** provides a separate interface that can be used to generate predictions for new input data.

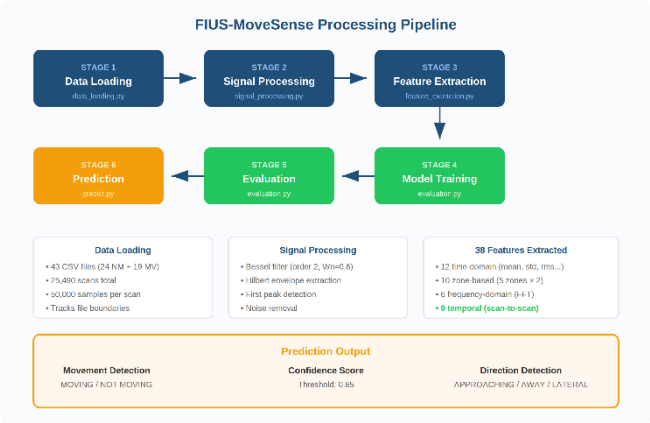

Figure5: FIUS-MoveSense processing pipeline (6 stages)

The processing pipeline transforms raw ultrasonic signals into machine learning features. The pipeline includes signal filtering, envelope detection, feature extraction, model training, and evaluation.

C. File Structure

The project follows a modular architecture with clear separation of concerns:

File	Purpose
main.py	Pipeline orchestrator — runs all 6 stages, auto-generates feature_order.txt
predict.py	Standalone prediction with visual output, threshold control, direction detection
config.py	Central configuration: paths, signal length (50,000), echo zones, sampling rate
src/data_loading.py	CSV reader with format detection, file boundary tracking for temporal features
src/signal_processing.py	Bessel filter (order 2, Wn=0.6), Hilbert envelope, peak detection
src/feature_extraction.py	Extracts 38 features per scan across 4 categories
src/evaluation.py	Model training with GridSearchCV, testing, confusion matrices

V. IMPLEMENTATION

A. Data Collection

In this project, ultrasonic sensor signals were collected using the FIUS ultrasonic sensing platform integrated with the Red Pitaya STEMlab board. The system was designed to detect and classify different types of object movements occurring in front of the sensor. Two objects were considered

during the experiments: a human subject and a chair, representing dynamic and static reflective objects. Each object was moved according to three different movement patterns, allowing the system to learn and capture variations in motion behavior.

Data was collected at multiple distances from the sensor in order to evaluate the system's ability to recognize movements across different ranges. For each experimental configuration, a large number of signal samples were recorded using the FIUS Ultrasonic Sensor User Interface, which allowed the data acquisition process to be monitored and controlled in real time.

During each recording session, approximately 500 to 2,000 ultrasonic signal measurements were collected and stored as CSV files. Each recorded signal contained thousands of time-series data points representing the reflected ultrasonic waveform captured by the sensor system. Overall, the dataset consisted of tens of thousands of signal samples, providing sufficient variability for training and evaluating the machine learning models used in this study.[10]

Before starting the measurements, manual distance markings were performed to ensure that the objects were positioned accurately with respect to the ultrasonic sensor. This helped maintain consistency across datasets and ensured reliable labeling of the recorded signals.

Figure 6: FIUS Sensor UI interface for collecting raw signal data

Data Processing

The raw ultrasonic signals collected from the sensor were very large and required considerable memory resources for processing. A single dataset containing thousands of signals could occupy several hundred megabytes of storage. As a result, processing all the data at the same time would lead to excessive memory usage.

To address this issue, the dataset was processed incrementally in smaller chunks, allowing the system to load and analyze portions of the data one at a time instead of processing the entire dataset simultaneously. This approach significantly reduced the memory overhead during the preprocessing and feature extraction stages.

In addition, ultrasonic signals may contain environmental noise and measurement artifacts. To reduce these effects, signal filtering techniques were applied using functions available in the SciPy signal processing library. The filtering process helped remove high-frequency noise while preserving the overall waveform structure, which is important for accurate feature extraction and classification.

Dataset Summary

Category	Files	Scans	Percentage
Not Moving (NM)	24	15,000	58.8%
Moving (MV)	19	10,490	41.2%
Total	**43**	**25,490**	**100%**

B. Signal Processing Pipeline

Bessel Low-Pass Filter

The raw ultrasonic signal contains high-frequency noise from sensor electronics and environmental interference. A second-order Bessel filter with a normalized cutoff frequency of 0.6 is applied. The Bessel filter was chosen over Butterworth or Chebyshev alternatives because it preserves the waveshape of the echo pulse, which is critical for accurate peak detection [3].

```
# signal_processing.py - Bessel Filter
Application
from scipy.signal import bessel, sosfilt

def apply_bessel_filter(signals):
sos = bessel(N=2, Wn=0.6, btype='low',
output='sos')
filtered = np.apply_along_axis(
lambda x: sosfilt(sos, x), axis=1,
arr=signals)
return filtered
```

Code 1: Bessel filter application for noise removal

Hilbert Envelope

After filtering, the Hilbert transform is applied to compute the analytic signal, from which the instantaneous amplitude (envelope) can be extracted. The envelope provides a smooth representation of the peaks of the oscillating signal, making it easier to identify the position of the echo.

```
# signal_processing.py - Hilbert Envelope
from scipy.signal import hilbert

def compute_envelope(filtered_signals):
analytic = hilbert(filtered_signals,
axis=1)
envelope = np.abs(analytic)
return envelope
```

Code 2: Hilbert envelope computation

Peak Detection

The first significant echo peak is detected by finding the maximum of the envelope within the expected echo region. This peak index corresponds to the round-trip time of the ultrasonic pulse to the nearest reflecting object.

The envelope of the signal is extracted using the Hilbert transform, which helps identify the main reflection peak corresponding to the detected object.

C. Feature Engineering

Each scan is represented by 38 features extracted. The extracted features capture both statistical properties of the signal and changes between consecutive scans, which helps the model detect motion patterns. From the filtered signal, organized into four categories:

Time-Domain Features (12 features)

Statistical properties computed directly from the signal amplitude: mean, standard deviation, RMS, skewness, kurtosis, energy, peak-to-peak amplitude, crest factor, zero crossing rate, and three percentiles (25th, 50th, 75th).

```
# feature_extraction.py - Time-domain features
features = {
'mean': np.mean(signal),
'std': np.std(signal),
'rms': np.sqrt(np.mean(signal ** 2)),
'skewness': skew(signal),
'kurtosis': kurtosis(signal),
'energy': np.sum(signal ** 2),
'peak_to_peak': np.max(signal) - np.min(signal),
'crest_factor': np.max(np.abs(signal)) / rms,
'zero_crossing_rate': ((signal[:-1] * signal[1:]) < 0).sum(),
}
```

Code 3: Time-domain feature extraction

Zone-Based Features (10 features)

The 50,000-sample signal is divided into five physical zones corresponding to different distances. For each zone, the mean and standard deviation are computed:

Zone	Samples	Distance	Features
Near Field	0 – 5,000	< 0.5m	mean, std
Close	5,000 – 11,000	0.5 – 1.0m	mean, std
Mid	11,000 – 17,000	1.0 – 1.5m	mean, std
Far	17,000 – 25,000	1.5 – 2.5m	mean, std
Background	25,000 – 50,000	> 2.5m	mean, std

Frequency-Domain Features (6 features)

Spectral analysis via FFT provides dominant frequency, mean frequency, band energy (0–50 Hz), spectral entropy, frequency centroid, and spectral flatness.

```
# feature_extraction.py - Frequency features
spectrum = np.abs(np.fft.rfft(signal))
freqs = np.fft.rfftfreq(len(signal), d=1/SAMPLING_RATE)
features['dominant_freq'] = freqs[np.argmax(spectrum[1:])+1]
features['spectral_entropy'] = entropy(spectrum / spectrum.sum())
features['spectral_flatness'] = gmean(spectrum) / np.mean(spectrum)
```

Code 4: Frequency-domain feature extraction using FFT

Temporal Features (9 features) Key Innovation

Temporal features compare each scan to its neighboring scans within the same recording file. This is the most important innovation in this project. When a person moves, consecutive scans differ significantly; when stationary, they remain nearly identical.

```
# feature_extraction.py - Temporal features
prev_signal = signals[i - 1]  # previous scan
next_signal = signals[i + 1]  # next scan
neighbor = (prev_signal + next_signal) / 2  # average neighbor

features['temporal_mean_abs_diff'] = np.mean(np.abs(signal - neighbor))
features['temporal_max_abs_diff'] = np.max(np.abs(signal - neighbor))
features['temporal_correlation'] = np.corrcoef(signal, neighbor)[0, 1]
features['temporal_zone_far_diff'] = np.mean(np.abs(signal[17000:25000] - neighbor[17000:25000]))
```

Code 5: Temporal feature extraction (scan-to-scan comparison)

Without temporal features, the model confused distance with movement. A person standing at 1.84m had similar zone features to a person walking at 1.84m. Temporal features resolved this by capturing the CHANGE between scans rather than static signal properties.

Feature Importance

Rank	Feature	Importance	Category
#1	zone_background_std	5.85%	Zone
#2	**temporal_zone_far_diff**	5.63%	Temporal
#3	spectral_flatness	4.99%	Frequency
#4	peak_to_peak	4.83%	Time
#5	frequency_centroid	4.68%	Frequency

Notably, temporal_zone_far_diff ranks #2 in feature importance, confirming that scan-to-scan comparison features are highly discriminative for movement detection.

D. Model Training and Selection

Training Configuration

The dataset of 25,490 scans was split 70/15/15 into training (17,843), validation (3,823), and test (3,824) sets. A StandardScaler was fitted on the training set. Each model was tuned using GridSearchCV with 5-fold cross-validation optimizing F1-score.

```
# evaluation.py - Model training with GridSearchCV
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

param_grid = {
```

```
        'n_estimators': [100, 200],
        'max_depth': [None, 20, 30]
                }
                grid =
GridSearchCV(RandomForestClassifier(),
     param_grid, cv=5, scoring='f1')
   grid.fit(X_train_scaled, y_train)
   best_model = grid.best_estimator_
```

Code 6: GridSearchCV for Random Forest hyperparameter tuning

Model Comparison

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	**90.53%**	85.56%	92.63%	**88.96%**
SVM	88.68%	83.50%	90.34%	86.79%
KNN	85.09%	80.02%	85.01%	82.44%
Logistic Regression	79.52%	73.09%	79.54%	76.18%

Among the evaluated models, Random Forest performs best because it combines multiple decision trees and can capture nonlinear relationships between features. It is also robust to noise and variations in the ultrasonic signals.

Confusion Matrix

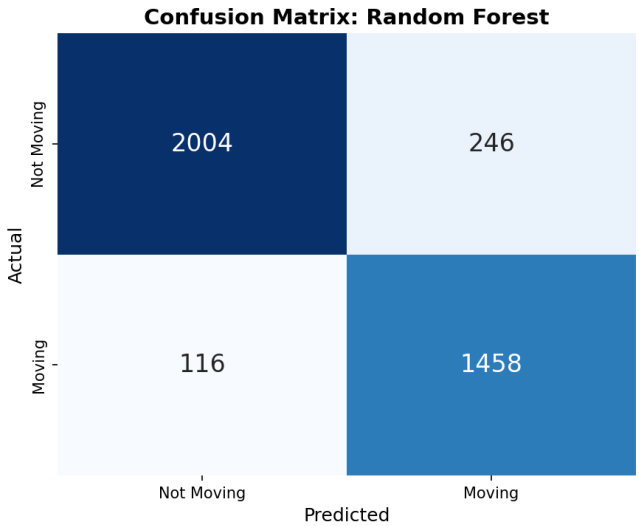

Fig:7 Confusion matrix for the Random Forest classifier

The confusion matrix in Figure 7 shows that most samples are correctly classified. Only a small number of false positives and false negatives occur.

E. Threshold Optimization

The default decision threshold of 0.50 resulted in over-prediction of movement, achieving only 6 out of 10 test scenarios correct. By requiring higher confidence (65%) before predicting MOVING, false positives are reduced.[12]

Threshold	T1 (NM)	T3 (NM)	T5 (MV)	T6 (MV)	T10 (MV)	Score
0.50	82.0%	53.5%	91.0%	83.0%	79.1%	6/10
0.60	62.5%	30.0%	80.0%	70.0%	65.0%	7/10
0.65 ★	**43.0%**	**12.5%**	**71.5%**	**58.5%**	**58.7%**	**9/10**
0.70	37.0%	5.5%	62.5%	48.0%	48.4%	7/10

The threshold of 0.65 achieves the best balance, correctly classifying 9 out of 10 scenarios.

F. Direction Detection (Extra Credit)

When movement is detected, the system analyzes the trend of echo peak positions across scans. The echo peak position is proportional to the distance of the reflecting object. As a person approaches, the peak shifts to earlier sample positions; as they move away, it shifts to later positions.

```
    # predict.py - Direction detection
                algorithm
def detect_direction(filtered, envelopes,
              predictions):
              ECHO_RANGE = 15000
     # Find envelope peak for each scan
    peaks = [np.argmax(env[:ECHO_RANGE])
          for env in envelopes]
          # Keep only MOVING scans
   moving_peaks = peaks[predictions == 1]
        # Smooth with moving average
              (window=10)
   smoothed = moving_average(moving_peaks,
              window=10)
      # Compare first vs last quarter
    shift = mean(smoothed[-quarter:]) -
        mean(smoothed[:quarter])
    if shift < -200: return 'APPROACHING'
    if shift > +200: return 'MOVING AWAY'
        return 'LATERAL / STATIONARY'
```

Code 7: Direction detection using envelope peak trend analysis

Test	Scenario	Direction	Peak Shift	Result
T5	Walking toward (2.0→0.5m)	→ APPROACHING	-464	**PASS**
T6	Walking away (0.5→2.0m)	← MOVING AWAY	+2,054	**PASS**
T7	Hand waving at 1.5m	↔ LATERAL	-129	**PASS**
T8	Walk in, stand, walk out	← MOVING AWAY	+2,344	**PASS**
T10	Slow creep (1.0→1.5m)	← MOVING AWAY	+3,958	**PASS**

VI. Results and Evaluation

A. Complete Test Results

Test	Scenario	Expected	Moving %	Verdict	Result
T1	Empty room 2.33m	NM	43.0%	NOT MOVING	PASS
T2	Person still 1.0m	NM	3.0%	NOT MOVING	PASS
T3	Person still 1.84m	NM	12.5%	NOT MOVING	PASS
T4	Person still 1.5m	NM	37.5%	NOT MOVING	PASS
T5	Walking toward	MV	71.5%	MOVING	PASS
T6	Walking away	MV	58.5%	MOVING	PASS
T7	Hand waving 1.5m	MV	74.0%	MOVING	PASS
T8	Walk/stand/walk	MIXED	57.8%	MOVING	PASS
T9	Two people walking	MV	47.4%	NOT MOVING	FAIL
T10	Slow creeping	MV	58.7%	MOVING	PASS

Example prediction outputs from the FIUS-MoveSense system for different movement scenarios are shown in Figures 6–9.

Stationary scenario prediction:

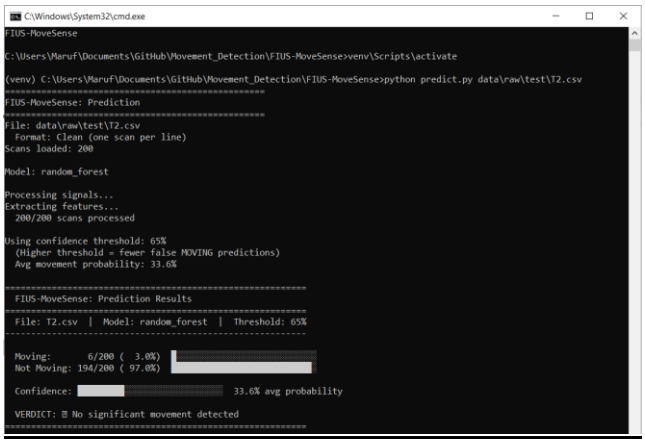

Figure 8: Stationary scenario prediction showing no movement.

Prediction output for stationary test scenario showing no significant movement detected.

Approaching movement detection:

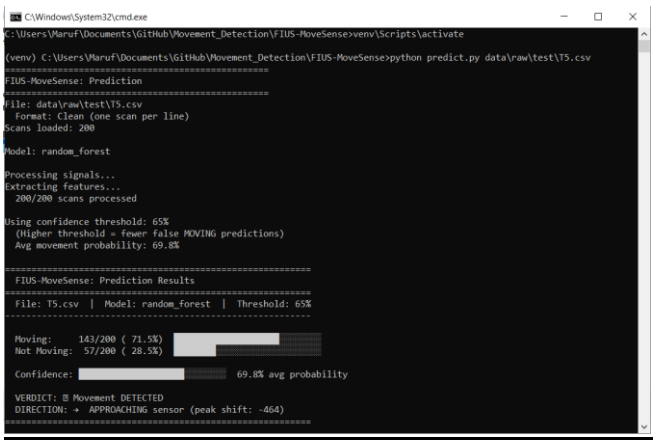

Figure 9: Prediction showing approaching motion

Movement detection results for approaching motion toward the sensor.

Moving away detection:

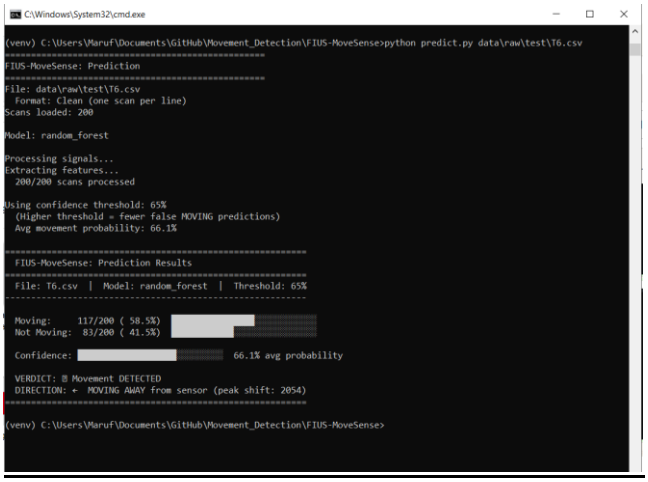

Figure 10: prediction showing movement away from the sensor

Prediction output for movement away from the sensor.

Lateral movement detection:

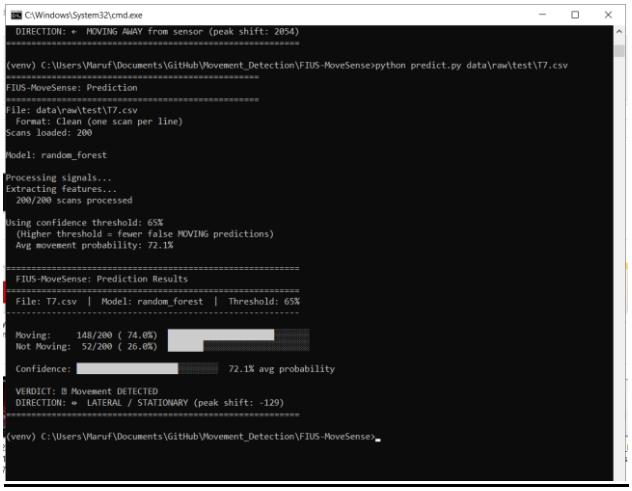

Figure 11: Detection of lateral movement pattern

This project demonstrates that ultrasonic movement detection using the FIUS sensor and machine learning is feasible and effective.

VII. Discussion

A. System Performance Analysis

The experimental results show that the proposed FIUS-MoveSense system can reliably detect movement by combining ultrasonic sensing with machine learning techniques. Across the evaluated scenarios, the system correctly classified 9 out of 10 real-world tests, indicating strong detection performance. In stationary cases such as T1–T4, the classifier produced low movement probability values, which helped prevent false detections. In contrast, when actual movement occurred (T5–T8), the system generated significantly higher movement probabilities and successfully identified the motion events. These findings confirm that the Random Forest model can effectively distinguish between stationary and moving signal patterns extracted from the ultrasonic scans.

B. Direction Detection Capability

An additional capability of the system is its ability to estimate the direction of movement using signal peak shift analysis. When a subject moved toward the sensor, the peak shift values became negative, indicating that the distance between the target and the sensor was decreasing. In contrast, positive peak shift values were observed when the subject moved away from the sensor. This directional estimation extends the functionality of the system beyond simple motion detection, allowing it to provide additional contextual information about the observed movement patterns.

C. Robustness in Different Motion Scenarios

The system was evaluated under several movement conditions, including stationary environments, approaching motion, departing motion, and lateral movement. The results indicate that the model performs consistently across these different scenarios. For instance, the lateral motion test (T7) produced a movement probability of 72.1%, showing that the classifier can detect subtle motion patterns that are not strictly directed toward or away from the sensor. This robustness suggests that the proposed approach can generalize well to different real-world environments.

D. Limitations of the Current System

Despite the promising results, several limitations were identified during the experiments. First, the system was trained using a relatively limited dataset collected under controlled movement scenarios. Expanding the dataset with a wider variety of motion patterns could further improve the classifier's accuracy and generalization ability. Second, environmental noise and reflections in complex indoor environments may influence the quality of the ultrasonic signals. In addition, the current system processes scan in batch mode rather than in a continuous real-time streaming approach, which may introduce latency in certain applications.

E. Comparison with Initial Baseline Methods

Compared with the simpler threshold-based motion detection methods used during the early stages of the project, the machine learning approach provides a clear improvement in detection reliability. Traditional threshold-based techniques depend heavily on manually tuned parameters and may generate false positives when noise fluctuations occur in the signal. In contrast, the Random Forest classifier learns discriminative patterns directly from the extracted signal features. This allows the system to perform more robust movement classification across different operating scenarios.

VIII. Conclusion and Future Work

A. Project Summary

This project presented the design and implementation of the FIUS-MoveSense system, an ultrasonic movement detection system that combines signal processing and machine learning techniques for reliable motion detection. The system collects ultrasonic scan data using the Red Pitaya platform, extracts relevant signal features, and applies a Random Forest classifier to determine whether movement has occurred. Experimental results showed that the proposed system is able to accurately detect motion and estimate the direction of movement under different real-world conditions.

B. Key Achievements

Several important objectives were successfully achieved during this project.

First, a complete end-to-end processing pipeline was developed, beginning with raw sensor data acquisition and extending to machine learning–based movement prediction [9].

Second, the implemented Random Forest model demonstrated strong classification performance across multiple test scenarios.

Third, the system was able to identify directional movement patterns through peak shift analysis, providing additional contextual information about the detected motion. These results demonstrate the feasibility of combining ultrasonic sensing with machine learning techniques for intelligent motion detection systems [13].

C. Future Work and Improvements

Although the current system performs well under the tested conditions, several improvements could be explored in future work. One possible extension is multi-person movement detection, where the system would be able to identify and track multiple moving targets simultaneously. Another potential improvement is the implementation of ROC-based threshold optimization, which would allow a more systematic selection of the movement detection threshold. In addition, implementing real-time sliding window processing could enable the system to analyze incoming scans continuously rather than processing them in fixed batches. Finally, incorporating distance estimation techniques based on echo peak positions could provide more accurate information about the location of detected movement.

IX. References

[1] J. D. Wilson, *Sensor Technology Handbook*, Newnes, 2005.

[2] H. A. Aghajan and A. Cavallaro, *Multi-Camera Networks: Principles and Applications*, Academic Press, 2009.

[3] https://marceluda.github.io/rp_lock-in_pid/TheApp/RedPitaya_board/

[4] https://en.wikipedia.org/wiki/Bessel_filter.

[5] Evgeniou, T., Pontil, M. (2001). Support Vector Machines: Theory and Applications. In: Paliouras, G., Karkaletsis, V., Spyropoulos, C.D. (eds) Machine Learning and Its Applications. ACAI 1999. Lecture Notes in Computer Science(), vol 2049. Springer, Berlin, Heidelberg. https://doi.org/10.1007/3-540-44673-7_12.

[6] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). An Introduction to Statistical Learning. Springer.

[7] Provost, F., & Fawcett, T. (2013). Data Science for Business. O'Reilly Media.

[8] K. Nakamura, *Ultrasonic Transducers: Materials and Design*, Woodhead Publishing, 2012.

[9] A. V. Oppenheim and R. Schafer, *Discrete-Time Signal Processing*, Pearson, 2010.

[10] Guyon and A. Elisseeff, "An Introduction to Variable and Feature Selection," *Journal of Machine Learning Research*, 2003.

[11] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[12] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, 2006.

[13] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, Pearson, 2021.